

PushAround: Collaborative Path Clearing via Physics-Informed Hybrid Search

Abstract—The passage of large vehicles in cluttered environments is often blocked by movable obstacles, motivating the use of mobile robot teams to proactively clear traversable corridors. Existing approaches to Navigation Among Movable Obstacles (NAMO) mainly plan abstract sequences of obstacle displacements but often neglect physical feasibility, overlooking robot dimensions, obstacle mass, contact geometry, and required forces, which can lead to strategies that fail in execution. This work introduces *PushAround*, a physics-informed hybrid search framework that incrementally constructs executable clearing plans by coupling geometric connectivity reasoning with short-horizon simulation. During search, repeated W-clearance connectivity graph (WCCG) queries expose frontier gaps where the vehicle corridor must be widened and rank them using estimated downstream gap-sequence costs. Guided by the selected gap, the search generates direct or recursive pushing candidates by choosing which obstacles to displace, in which directions, and with which contact points and forces; only physics-validated candidates are then admitted into the clearing plan. This interleaved geometry–physics screening avoids exhaustive rollout over obstacle–action combinations while preserving physically grounded plan expansion. **Additional validation characterizes connectivity-query consistency and sensitivity to physics-model mismatch.** The framework is demonstrated through extensive simulations and hardware experiments.

I. INTRODUCTION

In many real-world settings, the passage of large vehicles or transport units is obstructed by movable obstacles such as pallets, boxes, or equipment, motivating the deployment of mobile robot teams to actively clear traversable corridors and enable safe passage. This capability is critical in cluttered and unstructured environments such as warehouses, disaster sites, and dense urban spaces. Conventional path planning methods assume static obstacles and thus fail once direct routes are blocked [1]. Navigation Among Movable Obstacles (NAMO) has been studied as a means of creating new routes by pushing, pulling, or rotating obstacles [2], yet most methods abstract away physical feasibility. Factors such as robot dimensions, obstacle size and mass, contact geometry, applied forces, and the coupled dynamics of pushing are typically ignored, producing plans that are *geometrically valid but physically unrealizable*. As shown in Fig. 1, this difficulty is amplified when a large robot cannot reach obstacle boundaries, while a small robot lacks sufficient pushing force. Cooperative pushing by several small robots is a viable alternative, but introduces strong physical coupling since simultaneous pushes can trigger chained motions of multiple obstacles, making prediction and planning substantially more complex.

A. Related Work

NAMO extends motion planning to cluttered environments by planning obstacle displacements through pushing, pulling,



Fig. 1. Collaborative path clearing in both simulation (**Top**) and hardware (**Bottom**). Two robots actively push movable obstacles to create a *W*-clear path for a large vehicle to traverse from start to goal, via a hybrid search over the sequence of obstacles to push and the associated pushing modes.

or rotating [2], [3]. Later work considers heuristic search, learning-based local planning, physics-based rollouts, and collaborative pushing [3], [4], [5], [6], [7]. However, many NAMO formulations still simplify robot geometry, actuation limits, object mass, and contact dynamics, leading to physically unrealizable plans.

By employing multiple robots, collaborative pushing provides a potential solution for such scenarios and has been widely studied in multi-robot manipulation. Foundational studies examined the mechanics of pushing and the limit-surface model [8], [9]. Subsequent work considered cooperative transport with robot swarms [10], multi-robot box pushing with reinforcement learning [11], collaborative pushing of polytopic objects in cluttered scenes [7], and learning-based loco-manipulation for long-horizon quadrupedal pushing [12]. These studies demonstrate effective multi-robot cooperation, but typically focus on a prescribed object rather than multi-object corridor clearing.

Physics-informed planning has emerged to incorporate dynamics models, contact reasoning, or simulation feedback into motion generation. Learned dynamics have been used for humanoid contact planning [13], while multi-contact force-control methods address physically consistent execution [14]. For non-prehensile manipulation, demonstration-guided optimal control reasons over contact modes and pushing dynamics [15], and physics-based simulation is interleaved with multi-agent pathfinding for manipulation among movable objects [16], [17]. Neural motion-planning methods further use learned physical priors for motion generation [18], [19]. However, they do not directly address collaborative path clearing with many interacting movable obstacles.

B. Our Method

As shown in Fig. 2, this work introduces **PushAround**, a physics-informed framework for collaborative multi-robot

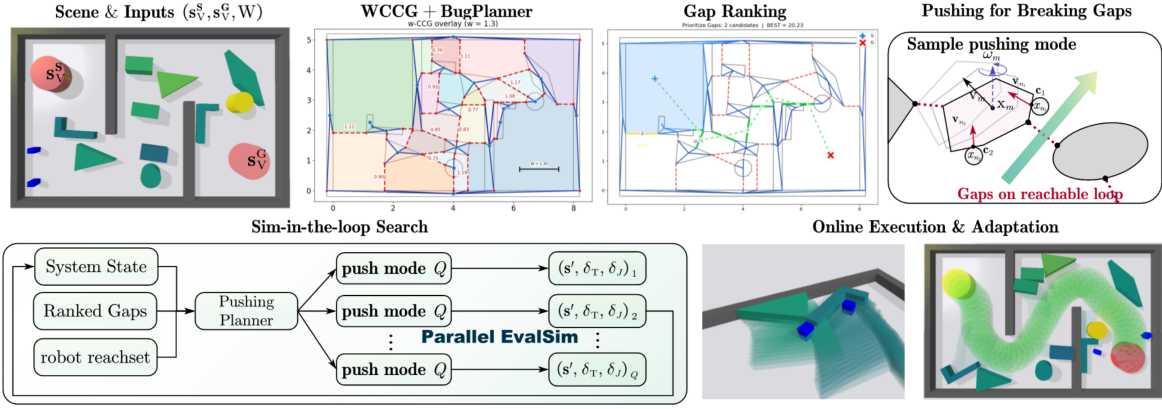


Fig. 2. Overview of the proposed framework. **Top:** Scene inputs, WCCG construction, gap ranking, and sampling of pushing modes for breaking frontier gaps; **Bottom-left:** Sim-in-the-loop hybrid search with parallel validation in simulation; **Bottom-right:** Online execution and adaptation during path clearing.

pushing that actively constructs traversable corridors for large vehicles in cluttered environments. The central novelty is a hybrid search that jointly determines which obstacles to displace and how to push them, coupling the high-level sequence of obstacle clearing with the low-level pushing modes that specify contact points, directions, and forces. The framework integrates three components: a W -clearance connectivity graph (WCCG) that certifies vehicle passage under width constraints, a gap-ranking strategy that prioritizes frontier gaps by estimated effort, and the pushing mode generation under geometric constraints. These components are encapsulated under a search procedure that incrementally expands candidate clearing plans while validating their physical realizability. This integration yields plans that are not only geometrically valid but also feasible under robot dimensions, object mass, contact geometry, and coupled pushing dynamics. [Since physics validation depends on model fidelity, robustness under execution-model mismatch is also evaluated.](#) Efficiency is achieved by combining compact geometric reasoning with prioritized evaluation. Extensive simulations and experiments are conducted for validation.

The contributions are twofold: (I) it introduces a unified framework that couples collaborative pushing with physics-informed verification, producing executable plans for path clearing in cluttered environments; and (II) it shows significant improvements in feasibility, efficiency and scalability over existing NAMO and collaborative clearing methods.

II. PROBLEM DESCRIPTION

A. Workspace and Robots

The workspace is a bounded planar region $\mathcal{W} \subset \mathbb{R}^2$ that contains two types of obstacles. A set $\mathcal{O} \subset \mathcal{W}$ represents immovable structures, while a set of $M \geq 0$ movable rigid polygons is defined as $\Omega \triangleq \{\Omega_1, \dots, \Omega_M\} \subset \mathcal{W}$. Each movable obstacle Ω_m is characterized by its mass M_m , inertia I_m , frictional parameters, and state $\mathbf{s}_m(t) \triangleq (\mathbf{x}_m(t), \psi_m(t))$, where $\mathbf{x}_m(t) \in \mathbb{R}^2$ is the planar position of its centroid and $\psi_m(t) \in \mathbb{R}$ its orientation. The region occupied by Ω_m at time t is denoted $\Omega_m(t)$. Moreover, a small team of N robots, indexed as $\mathcal{R} \triangleq \{R_1, \dots, R_N\}$, operates as a cooperative unit. Each robot R_i is modeled as a rigid body with state $\mathbf{s}_{R_i}(t) \triangleq (\mathbf{x}_{R_i}(t), \psi_{R_i}(t))$, where $\mathbf{x}_{R_i}(t) \in \mathbb{R}^2$ is its position

and $\psi_{R_i}(t) \in \mathbb{R}$ its orientation. The footprint of R_i is denoted $R_i(t)$. The instantaneous free space is given by:

$$\widehat{\mathcal{W}}(t) \triangleq \mathcal{W} \setminus \left(\mathcal{O} \cup \{\Omega_m(t)\}_{m=1}^M \cup \{R_i(t)\}_{i=1}^N \right), \quad (1)$$

where $\widehat{\mathcal{W}}(t)$ excludes regions occupied by immovable obstacles, movable obstacles, and robots.

B. External Vehicle and Clearance Goal

An external vehicle $\mathcal{V}$ whose footprint is enclosed by a disk of radius $W/2 > 0$ must navigate within the workspace from a start position $\mathbf{x}_V^S \in \mathcal{W}$ to a goal position $\mathbf{x}_V^G \in \mathcal{W}$. Since movable obstacles may obstruct the way, the clearance goal is to rearrange them so that the workspace contains a passage from $\mathbf{x}_V^S$ to $\mathbf{x}_V^G$ that can accommodate this enclosing disk, i.e., a passage of width at least W . At time t , this is expressed by the admissible reference-center space $\mathcal{F}_W(t) \triangleq \{p \in \widehat{\mathcal{W}}(t) \mid \mathbb{B}_{W/2}(p) \subset \widehat{\mathcal{W}}(t)\}$, where $\mathbb{B}_{W/2}(p)$ is the disk of radius $W/2$ centered at p . The W -clearance condition is then given by:

$$\exists \mathcal{P}_V^W \subset \mathcal{F}_W(t) : \mathbf{x}_V^S \rightsquigarrow \mathbf{x}_V^G, \quad (2)$$

which certifies a collision-free centerline path for any vehicle footprint contained in the enclosing disk.

C. Collaborative Pushing Modes

To enlarge blocked gaps, the robots may actively reconfigure Ω by pushing movable obstacles. For obstacle $\Omega_m \in \Omega$, a pushing mode is denoted by $\xi_m \triangleq (\mathbf{c}_m, \mathbf{u}_m)$, where $\mathbf{c}_m = (c_1, \dots, c_N)$ is an ordered tuple of contact points on $\partial\Omega_m$, and the ordering specifies the robot-to-contact assignment. The vector $\mathbf{u}_m \in \mathbb{R}^{2N}$ stacks the corresponding planar contact forces, expressed in the body frame of Ω_m . For a contact tuple $\mathbf{c}_m$, the set of admissible forces $U_m(\mathbf{c}_m) \subset \mathbb{R}^{2N}$ collects contact forces satisfying the robot pushing limits, robot-object friction, and unilateral contact constraints. The body wrench induced by a mode is defined as:

$$\mathbf{q}_m(\xi_m) \triangleq G_m(\mathbf{c}_m)\mathbf{u}_m, \quad (3)$$

where $G_m(\mathbf{c}_m) \in \mathbb{R}^{3 \times 2N}$ is the contact-to-wrench mapping; and $\mathbf{q}_m(\xi_m) \in \mathbb{R}^3$. The admissible mode set is state dependent as denoted by $\Xi_m(\mathbf{s})$. It contains modes whose contacts are reachable from the current robot configurations, pre-contact

configurations are feasible in the current clutter, and forces satisfy $\mathbf{u}_m \in U_m(\mathbf{c}_m)$. Hence, applying a feasible mode $\xi_m \in \Xi_m(\mathbf{s}(t))$ yields the physical transition:

$$\mathbf{s}(t^+) \triangleq \Phi(\mathbf{s}(t), m, \xi_m), \quad (4)$$

where $\mathbf{s}(t)$ stacks all robot and obstacle states. The force vector $\mathbf{u}_m$ serves as a feasibility variable to certify wrench generation under the modeled contact and actuation constraints rather than a directly commanded control input.

D. Problem Statement

The goal is to compute a *hybrid schedule* that reconfigures movable obstacles to admit a W -feasible path for the external vehicle. The schedule is defined as:

$$\pi \triangleq \{(\Omega_{m_k}, \xi_k, \Delta t_k)\}_{k=1}^K, \quad (5)$$

where $\Omega_{m_k} \in \Omega$ is the manipulated obstacle, $\xi_k \in \Xi_{m_k}$ is its pushing mode, and $\Delta t_k > 0$ is the execution duration. The search is guided by

$$\mathcal{J}_{\text{clr}}(\pi) \triangleq T + \alpha \sum_{k=1}^K J(m_k, \xi_k, \mathbf{s}(t_k)), \quad (6)$$

where $T \triangleq \sum_{k=1}^K \Delta t_k$ denotes the robotic clearance duration excluding subsequent vehicle traversal, and $J(\cdot)$ is the simulation-estimated manipulation effort. Vehicle traversal is considered only through the terminal W -clearance condition (2). The constraints (1)–(4) enforce valid pushing transitions. Equation (6) ranks feasible schedules; PIHS uses finite sampling and does not guarantee global optimality of the continuous hybrid objective.

Remark 1. The objective and constraints above couple obstacle selection, pushing-mode generation, and switching in a single hybrid planning problem. The resulting combinatorial-continuous search space is substantially richer than classical NAMO formulations with simple objects or hand-crafted contact modes [8], [10], [11]. ■

III. PROPOSED SOLUTION

As shown in Fig. 2, PushAround first uses the W -Clearance Connectivity Graph in Sec. III-A and gap ranking in Sec. III-B to identify and prioritize blocking gaps. It then generates and evaluates physically feasible collaborative pushing modes in Sec. III-C. These components are integrated into the simulation-in-the-loop hybrid search in Sec. III-D, followed by the online execution procedure summarized in Sec. III-E.

A. W -Clearance Connectivity Graph

The feasibility of routing the vehicle reduces to whether a W -width corridor connects $\mathbf{x}_V^S$ and $\mathbf{x}_V^G$. A W -clearance connectivity graph (WCCG) is introduced here to encode obstacle adjacencies under clearance W and enables fast encodes obstacle adjacencies under clearance W and enables fast connectivity queries. Constructed directly from obstacle geometry rather than a workspace grid, the WCCG does not introduce a grid-resolution parameter in the repeated connectivity queries used during planning.

1) *Graph Construction:* The WCCG is built by decomposing all clearance-relevant obstacle and workspace boundaries into convex components or segments. From each component C , a centroid node v_c is created. When two components C_u and C_v have closest points $p_u \in C_u$ and $p_v \in C_v$ with distance less than W , bridge nodes are added at p_u and p_v . These bridge nodes are connected by a bridge-bridge edge, annotated with the gap width $w_{uv} \triangleq \|p_u - p_v\|$, and further linked back to their centroids with centroid-bridge edges. Narrow passages with $w_{uv} < W$ are marked as potential bottlenecks. The resulting graph is given by:

$$\mathcal{G}_W \triangleq (\mathcal{V}, \mathcal{E}_c \cup \mathcal{E}_b), \quad (7)$$

where $\mathcal{V}$ are centroid and bridge nodes; $\mathcal{E}_c$ contains centroid-bridge edges; and $\mathcal{E}_b$ for bridge-bridge edges with widths w_{uv} .

2) *Connectivity Criterion:* Once $\mathcal{G}_W$ has been constructed, connectivity queries can be performed without explicitly computing a geometric path. Let $\text{Face}(\mathcal{G}_W)$ denote the set of connected planar faces induced by the embedded WCCG, i.e., the maximal regions separated by obstacle components and WCCG bottleneck edges. The criterion for (2) is given by:

$$(\mathbf{x}_V^S, \mathbf{x}_V^G \in \mathcal{F}_W(t)) \text{ and } (\exists F \in \text{Face}(\mathcal{G}_W) : \mathbf{x}_V^S, \mathbf{x}_V^G \in F), \quad (8)$$

The criterion follows the standard configuration-space inflation interpretation: expanding obstacles by a disk of radius $W/2$ turns gaps narrower than W into overlapping barriers. Non-convex obstacles are handled through clearance-relevant decomposition, after which WCCG records the separating bottlenecks. The consistency between WCCG connectivity and rasterized configuration-space connectivity is further evaluated in Sec. IV-A3. A frontier-tracing procedure, similar to the BugPlanner [20], is then used to extract a skeleton Σ as an ordered sequence of centroid and bridge nodes. Starting from $\mathbf{x}_V^S$, it casts a ray toward $\mathbf{x}_V^G$ and follows the encountered frontier loop until either a valid exit is found or a blocking cycle is returned.

B. Ranking of Frontier Gaps

When the condition in (8) fails, the vehicle cannot reach $\mathbf{x}_V^G$ from $\mathbf{x}_V^S$ through a W -clear path $\mathcal{P}_V^W$. In this case, the planner must prioritize *blocking gaps* on the reachable frontier of $\mathcal{G}_W$ according to their predicted downstream cost. The ranked list provides the local clearance targets that guide the hybrid search in the sequel.

1) *Frontier Extraction and Candidate Gaps:* A BugPlanner query on $\mathcal{G}_W$ above either confirms connectivity or returns a counter-clockwise frontier loop $\mathcal{L}$ that separates the two endpoints. The bridge-bridge edges on $\mathcal{L}$ define the first-hop candidate set $\Gamma_{\mathcal{L}} \triangleq \{g_1, \dots, g_K\}$, where each gap g_k is represented by its two closest boundary points, its width $w_{g_k} < W$, and the adjacent obstacles that can potentially be moved. These candidates are then ranked by predicted downstream cost and passed to the hybrid search as local clearance targets.

2) *Evaluation for Immediate Cost:* Each candidate $g \in \Gamma_{\mathcal{L}}$ is evaluated by combining the cost for the robots to reach the gap and the effort required to widen it. Let $\mathbf{s}_{\mathcal{R}}$ be the current

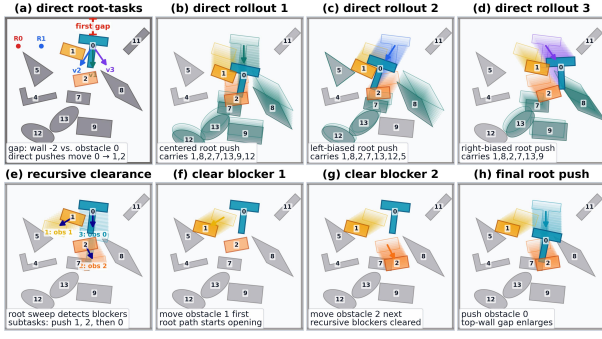


Fig. 3. **Direct** pushing moves a gap-side blocker along a gap-opening direction, whereas **recursive** pushing first clears movable blockers in the swept region and then returns to the root gap-opening task.

robot positions, and $\mathbf{o}_g$ as the outside insertion point of gap g . The resulting one-hop cost is given by:

$$C(g | \mathcal{L}, \mathbf{s}_R) = \lambda_t C_t(\mathbf{s}_R, \mathbf{o}_g) + \lambda_p (C_{dp}(g) + C_{rp}(g)) \quad (9)$$

where $\lambda_t, \lambda_p > 0$ weight transition and pushing costs, respectively; C_t , C_{dp} , and C_{rp} denote transition, direct-pushing, and recursive-pushing costs.

3) *Long-term Cost w.r.t. Goal*: Furthermore, to prioritize gaps closer to the goal, a heuristic is added to the evaluation, i.e., $h(g) = \eta \|\mathbf{o}(g) - \mathbf{x}_V^G\|$, with the scaling constant $\eta > 0$. Lastly, a short A*-style presearch is adopted to cross each candidate gap, recompute the next frontier, and continue for a limited width and depth. The *cost-to-connect* is predicted by:

$$\widehat{\text{Cost}}(g) \triangleq C(g | \mathcal{L}, \mathbf{s}_R) + \sum_{g' \in \Pi^*(g)} \{C(g' | \cdot) + h(g')\}, \quad (10)$$

where $\Pi^*(g)$ is the sequence of subsequent gaps discovered after virtually crossing g . Consequently, the final output of this module is the ranked list of candidate gaps, i.e., $\text{Rank}(\Gamma_{\mathcal{L}}) \triangleq \text{argsort}_{g \in \Gamma_{\mathcal{L}}} \widehat{\text{Cost}}(g)$, in ascending predicted cost. This ordered set is passed to the hybrid search module, such that only the promising gaps are expanded first.

C. Pushing Mode Generation and Evaluation

For a selected frontier gap g , this subsection constructs a set of candidate pushing strategies $\Xi_g \triangleq \{(\Omega_m, \mathbf{v}, \xi)\}$, where $\mathbf{v} \triangleq (v_x, v_y, \omega) \in \mathbb{R}^3$ is a short-horizon pushing direction for the movable obstacle Ω_m induced by g , and $\xi \triangleq (\mathbf{c}_m, \mathbf{u})$ is the associated pushing mode in (4).

1) *Direct and Recursive Pushing Tasks*: For *direct push*, let Ω_m denote one of the two movable obstacles adjacent to g . For each such obstacle, a small set of candidate body velocities $\{\mathbf{v}_m^k\}_{k=1}^{K_m}$ is sampled with a bias toward enlarging the gap g , while a few angular components are included to account for local geometry. However, if the swept region of a direct push intersects movable blockers, *recursive push* tasks are further required. Namely, for each out-layer blocker Ω_m , the nominal direction moves it away from both the swept region and the reference gap sequence $\Pi^*(g)$, with the candidate velocities $\{\mathbf{v}_m^k\}_{k=1}^{K_m}$. The recursive pushes aim to move the blocker out of the swept region rather than directly open the gap. Recursion therefore creates a bounded dependency chain which prioritizes the final blocker Ω_m , as shown in Fig. 3.

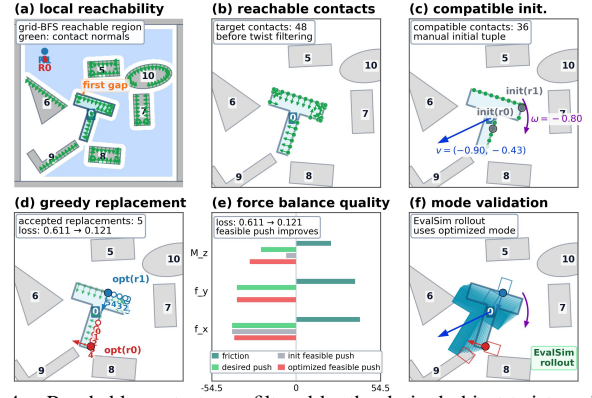


Fig. 4. Reachable contacts are filtered by the desired object twist, assigned to robots, refined by local replacement, and finally validated by simulation.

2) *Pushing Mode Generation*: At a search state $\mathbf{s}$, a pushing mode is generated for each candidate direction $\mathbf{v}_m^k$ and obstacle Ω_m . In this subsection, contact tuples, object velocities, forces, and wrenches are expressed in the body frame of Ω_m . For a contact tuple $\mathbf{c}$, its multi-directional force feasibility is:

$$J_{MF}^m(\mathbf{c}, \mathbf{v}_m^k) \triangleq \sum_{\mathbf{v}_d \in \mathcal{D}_m^k} w_d \min_{\mathbf{u} \in U_m(\mathbf{c})} \|\mathbf{q}_m(\mathbf{c}, \mathbf{u}) + \mathbf{q}_m^\mu(\mathbf{v}_d)\|, \quad (11)$$

where $\|\cdot\|$ is a weighted ℓ_1 wrench norm, $\mathcal{D}_m^k$ is a finite perturbation set around $\mathbf{v}_m^k$, and $\mathbf{q}_m^\mu(\mathbf{v}_d)$ is the quasi-static friction wrench of Ω_m ; the inner minimization is a small Linear Program (LP) over the admissible contact forces $U_m(\mathbf{c})$. A state- and direction-dependent contact set $\mathcal{A}_m^k(\mathbf{s}) \subset \partial\Omega_m$ is obtained from the robot-reachable boundary of Ω_m by velocity-force compatibility filtering, as illustrated in Fig. 4. Reachability is checked before mode optimization: a contact candidate is retained only if its assigned robots can reach collision-free pre-contact poses. Starting from cached layouts, if available, and random initializations in $\mathcal{A}_m^k(\mathbf{s})$, the contact tuple is refined by sequential greedy replacement. At the ℓ -th greedy step, let $\mathbf{c}^\ell$ be the current tuple, and let $\mathbf{c}_{r,c}^\ell$ be obtained by replacing its r -th contact with $\mathbf{c} \in \mathbb{R}^2$. The best replacement is derived by:

$$\mathbf{c}^* = \underset{\mathbf{c} \in \mathcal{A}_m^k(\mathbf{s})}{\text{argmin}} J_{MF}^m(\mathbf{c}_{r,c}^\ell, \mathbf{v}_m^k), \quad (12)$$

where the greedy refinement for $\mathbf{c}^*$ sweeps over contact slots until J_{MF}^m no longer decreases. Among initializations, the best local tuple is selected as $\mathbf{c}_m^{k,*}$, thus $\mathbf{u}_m^k$ is obtained by the inner minimization in (11) for $\mathbf{v}_m^k$. This yields $\xi_m^k = (\mathbf{c}_m^{k,*}, \mathbf{u}_m^k)$. The mode is cached as a warm start for similar future queries. The best few pairs $(\mathbf{v}_m^k, \xi_m^k)$ with $\xi_m^k \in \Xi_m(\mathbf{s})$ are retained for gap g and passed to simulation evaluation. The optimized force vector $\mathbf{u}_m^k$ certifies wrench feasibility; hardware executes pose/velocity commands that maintain the contact geometry.

3) *Pushing Strategy Evaluation*: For a current state $\mathbf{s}$ and a candidate strategy $(\Omega_m, \mathbf{v}, \xi) \in \Xi_g$, a short-horizon rollout evaluates the transition in (4) under the selected mode. As shown in Fig. 4, a strategy is accepted only if the robots reach the pre-contact poses, maintain stable contact, and induce obstacle motion above threshold. The simulation returns:

$$(\mathbf{s}', \delta_T, \delta_J) \triangleq \text{EvalSim}(\mathbf{s}, (\Omega_m, \mathbf{v}, \xi)), \quad (13)$$

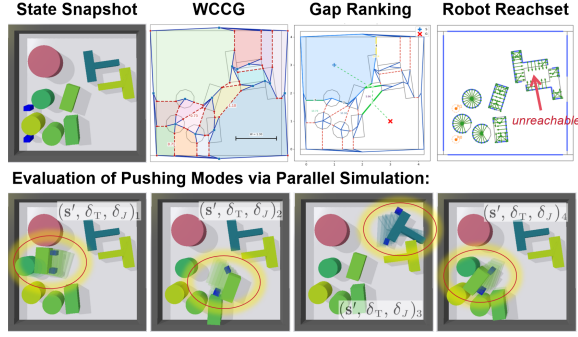


Fig. 5. **Top:** Core components of the proposed physics-informed hybrid search; **Bottom:** Sampled push tasks are evaluated via parallel simulation, producing updated states for search expansion.

where s' is the successor state capturing the resulting robot and object configurations; $\delta_T > 0$ is the time cost; and δ_J is the control effort. The rollout jointly simulates all robots and movable bodies, thereby capturing contact-induced object interactions. For efficiency, the pushing rollout may be bypassed when the mode has been validated before and the swept region of Ω_m is collision-free.

D. Physics-Informed Hybrid Search

The proposed physics-informed hybrid search (PIHS) selects blocking gaps and validates corresponding multi-robot pushing strategies through short-horizon simulation, as shown in Fig. 5 and summarized in Alg. 1. Unlike purely geometric NAMO planners, PIHS expands only physics-validated strategies through parallel candidate evaluation. It is a feasibility-oriented heuristic search, where Eq. (6) defines the ranking objective for candidate schedules. The search operates on a finite sampled candidate set and therefore does not guarantee global optimality or completeness of the underlying continuous hybrid planning problem.

1) *Tree and Initialization:* The search tree $\mathcal{T}$ is composed of nodes $\nu \triangleq (s, \pi)$, where s is the current system state of all robots and movables in (4), and π the partial pushing strategy in (6). Two global functions are maintained: $\text{Rank}(\nu)$ stores ranked candidate gaps $\Gamma_{\mathcal{L}_\nu}$ given the frontier loop $\mathcal{L}_\nu$ associated with node ν , together with their exploration costs by (10); and $\chi(\nu)$ returns the cumulative execution cost from the root to ν . The root is initialized as $\nu_0 \triangleq (s_0, \emptyset)$, where s_0 is the initial system state.

2) *Node Selection:* At each iteration, the node with minimum best-first priority is selected from the queue. Priority balances realized cost and estimated remaining effort:

$$f(\nu) \triangleq \chi(\nu) + \min_{g \in \text{Rank}(\nu)} \widehat{\text{Cost}}(g), \quad (14)$$

where $\chi(\nu)$ is the cumulative execution cost so far, and the second term is the minimum predicted remaining cost among the unexplored gaps of ν . The remaining-cost term is a heuristic and is not required to be admissible. The score prioritizes nodes by realized cost and promising future actions, but it does not provide an A* optimality guarantee.

3) *Node Expansion with Parallel Simulation:* Let the selected node be $\nu \triangleq (s, \pi)$. For each ranked gap $g \in \text{Rank}(\nu)$, its predicted gap sequence $\Pi^*(g)$ in (10) is first derived. Then,

Algorithm 1: Physics-Informed Hybrid Search

Input: $s_0, \alpha, \text{EvalSim}(\cdot), W$ -criterion by (8)
Output: π^*

```

1  $\nu_0 \leftarrow (s_0, \emptyset), \chi(\nu_0) = 0;$ 
2 Init  $\mathcal{Q}$  by (14);
3 while not terminated do
4    $\nu \leftarrow \mathcal{Q}.\text{pop\_min}()$  by (14);
5   foreach  $g \in \text{Rank}(\nu)$  do
6     Compute  $\Pi^*(g)$ , generate  $\Xi_g$ ;
7     foreach  $(\Omega_m, \mathbf{v}, \xi) \in \Xi_g$  do
8        $\tau = (\Omega_m, \mathbf{v}, \xi)$  // Direct or recur.
9        $(s', \delta_T, \delta_J) \leftarrow \text{EvalSim}(\nu, \tau)$  by (13);
10      if success then
11         $\nu' \leftarrow (s', \pi \cup \{\tau\});$ 
12         $\chi(\nu') \leftarrow \chi(\nu) + \delta_T + \alpha \delta_J;$ 
13        Add  $\nu'$  to  $\mathcal{Q}$ ;
14      if  $s'$  admits  $\mathcal{P}_V^W$  by (8) then
15         $\pi^* \leftarrow \pi',$  break;
16 return  $\pi^*;$ 

```

the mode optimization by (11) provides a candidate strategy set $\Xi_g \triangleq \{(\Omega_m, \mathbf{v}, \xi)\}$, including both the direct push of the adjacent movable obstacles, and recursive push of out-layer obstacles. Thus, expansion is prioritized first by the gap ordering in $\text{Rank}(\nu)$, and then by the candidate ordering within Ξ_g . Each candidate strategy $(\Omega_m, \mathbf{v}, \xi) \in \Xi_g$ is first checked by a geometric quick-pass and, if it passes, evaluated by the short-horizon rollout in (13). A successful evaluation yields (s', δ_T, δ_J) , from which a child node $\nu' \triangleq (s', \pi')$ is created by appending $\tau \triangleq (\Omega_m, \mathbf{v}, \xi)$ to the current partial schedule π . The cumulative cost is updated by: $\chi(\nu') \triangleq \chi(\nu) + \widehat{C}(\nu, \tau)$, where $\widehat{C}(\nu, \tau) \triangleq \delta_T + \alpha \delta_J$ is the realized incremental cost. Note that all candidate strategies in the current batch are evaluated *concurrently in parallel*, and a single expansion can produce multiple validated successors.

4) *Termination:* Termination occurs when a node ν reaches a state s for which a W -clear path $\mathcal{P}_V^W$ exists from $\mathbf{x}_V^S$ to $\mathbf{x}_V^G$ by (8). Thus, the schedule π stored in ν is a feasible solution satisfying the terminal clearance condition and the validated pushing transitions. It encodes both the sequence of gaps to clear and the pushing strategies that realize them, without implying that $\mathcal{J}_{\text{clr}}(\pi)$ is globally minimal.

Remark 2. Efficiency comes from three lightweight heuristics: gap-lookahead reduces unproductive branches, MODE-CACHE warm-starts mode optimization, and the quick-pass rollout bypass skips simple motions whose mode was previously validated, avoiding redundant simulations. ■

E. Overall Analyses

1) *Execution and Online Adaptation:* The hybrid search returns a pushing schedule $\pi = \{\tau_1, \dots, \tau_K\}$, where $\tau_k \triangleq (\Omega_k, \mathbf{v}_k, \xi_k)$ specifies the target obstacle, pushing direction and mode. The schedule is executed sequentially in a receding-horizon manner. For each τ_k , the robots move to the required mode ξ_k for obstacle Ω_k and follow the pushing direction $\mathbf{v}_k$



Fig. 6. Execution and planning alignment in Scenario 1 (Left) and Scenario 2 (Right). **Top**: simulated snapshots during path clearing. **Bottom**: corresponding WCCG overlays with the start face in blue. The executed pushes progressively expand the reachable W -clearance region until the goal is connected.

from the current state as in (13). On the hardware, optimized contact forces are used for mode feasibility rather than direct force commands; the controller converts the planned contact geometry and object motion into velocity commands with feedback tracking. During transition, robots are reassigned by preserving the circular order of contact points along the obstacle boundary [7], which reduces path crossings under sufficient clearance. After each push, the system state is updated from observation, and the execution proceeds to τ_{k+1} if the action succeeds. For consistency with execution, the WCCG is rebuilt periodically from the current configuration. Execution terminates once a W -clear path $\mathcal{P}_V^W$ connects $\mathbf{x}_V^S$ and $\mathbf{x}_V^G$. Replanning is triggered by observed execution discrepancies, including failed contact establishment, stalled pushing, or invalidated remaining WCCG connectivity. In such cases, Alg. 1 is called from the current state to derive the updated pushing schedule.

2) *Computational Complexity*: Given M movable obstacles, the computational cost of one node expansion in Alg. 1 is dominated by candidate generation and physics validation. The WCCG update and gap ranking incur $\mathcal{O}(M \log M + |\mathcal{V}| + |\mathcal{E}_c| + |\mathcal{E}_b|)$ and $\mathcal{O}(|\Gamma_{\mathcal{L}_\nu}| \log |\Gamma_{\mathcal{L}_\nu}|)$ costs, respectively, while the dominant simulation cost is

$$\mathcal{O} \left(\sum_{g \in \text{Rank}(\nu)} |\Xi_g| \left(C_{\text{LP}} + \frac{\rho_\nu T_{\text{sim}}}{n_w} \right) \right),$$

Therefore, scalability is mainly determined by validated pushing candidates, while quick-pass filtering and parallel simulation reduce the practical expansion cost. Robot-team size mainly affects mode generation, where greedy contact refinement avoids exhaustive $|\mathcal{A}|^N$ contact assignment and evaluates bounded single-contact replacements.

IV. EXPERIMENTS

Numerical experiments are conducted to validate the proposed PIHS algorithm. The algorithms are implemented in Python3 and PyBullet [21] on a computer with an Intel Core i9-14900KF and an NVIDIA RTX 4080. Videos are provided as multimedia attachments.

A. Numerical Simulations

1) *Setup and Configuration*: The simulation uses a step size of $\Delta t = 1/240$ s and a control period of $1/40$ s. Robots

are modeled as disk or box pushers consistent with the W -clearance definition, while movable obstacles have masses uniformly sampled from $[5, 15]$ kg. A trial succeeds when the external vehicle reaches its goal through a W -clear path. The nominal scenario uses an 8×5 m workspace with bottleneck-forming bar obstacles and randomly distributed movable objects. Two robots clear 10–15 movable obstacles under minimum-separation constraints. Unless otherwise stated, comparisons and ablations are conducted on Scenario 1 with 10 movable obstacles over 40 trials, while Scenario 2 with 14 obstacles provides an additional cluttered case. All methods use the same robot footprints, obstacle configurations, collision/reachability checks, and execution simulator.

2) *Results*: Two representative simulation runs are shown in Fig. 6. Scenario 1 contains 10 movable obstacles and two robots. In the initial state, the start and goal belong to different W -clearance faces, indicating that the target passage is blocked by movable obstacles. Guided by the selected frontier gaps, the planner obtains a clearing plan with 6 push tasks in 7.4 s. During execution, the planned pushes progressively enlarge the reachable W -clearance region. The execution finishes in 42.27 s, and the final WCCG overlay shows that the start face contains the goal. Meanwhile, Scenario 2 increases to 14 movable obstacles. The planner finds a 9-task clearing plan in 11.2 s, and the execution finishes in 61.27 s. Compared with Scenario 1, the longer task sequence reflects the additional bottlenecks and stronger obstacle interactions. In both scenarios, the updated WCCG overlays are aligned with the executed pushes: the blue start face expands after each clearance until a complete W -clear path is found.

3) *WCCG Connectivity Validation*: To validate WCCG connectivity, its prediction is compared with a configuration-space occupancy check over N randomly generated scenarios containing M movable obstacles. The WCCG achieves **XX%** agreement with **FP** false positives and **FN** false negatives, confirming that the abstraction preserves the connectivity relevant to W -clear path planning.

4) *Comparisons*: Four baselines are evaluated on Scenario 1 over 40 randomized trials under the same W -clearance criterion, object distribution, and simulator. All methods use the same pre-contact reachability checks. The baselines include **DFS-WCCG**, a WCCG-guided search with fixed pushes; **SL-Push (offline)**, geometric pushing without physics validation; **SL-Push (sim)**, its simulation-validated variant; and **Rec-NAMO**, a recursive NAMO planner [3], [4], [16], [5]. Table I shows that PushAround achieves the highest

TABLE I
PERFORMANCE COMPARISON AND ABLATIONS ON SCENARIO 1
OVER 40 RANDOMIZED TRIALS.

Method / Variant	Succ.(%)	PT (s)	ET (s)	#Sims	#Pushes
DFS-WCCG	25.0	>100.0	51.2	>500	8.0
SL-Push (offline)	62.5	0.06	44.1	0.0	7.3
SL-Push (sim)	75.0	18.2	64.6	20.0	10.2
Rec-NAMO	37.5	13.3	42.4	0.0	7.8
PushAround (ours)	97.5	10.3	28.6	121.5	6.0
w/o gap-lookahead	82.5	16.9	35.2	178.3	7.2
w/o mode-cache	92.5	17.1	33.4	164.6	6.5
w/o quick-pass	90.0	23.6	41.9	208.2	6.1
w/o reinsertion	85.0	12.8	31.1	151.7	7.8



Fig. 7. Failure cases by baselines Rec-NAMO, SL-Push and DFS-WCCG.

success rate of 97.5% and the lowest combined planning and execution time of 38.9 s. DFS-WCCG suffers from exponential branching, requiring more than 100 s of planning. SL-Push (offline) plans almost instantly in 0.06 s, but its non-physical displacements often become unrealizable during execution, resulting in 62.5% success. SL-Push (sim) improves success to 75.0% by validating pushes in simulation, but increases the planning time to 18.2 s and requires 10.2 pushes on average. Rec-NAMO often fails to complete a connected W-clear corridor in dense clutter.

5) *Ablation Studies*: Moreover, four internal ablations are conducted: **w/o gap-lookahead**, which ranks frontier gaps without downstream costs; **w/o mode-cache**, which disables warm starts from previously validated modes; **w/o quick-pass**, which evaluates all candidate strategies by rollout; and **w/o reinsertion**, which prevents promising but temporarily failed nodes from being reconsidered. Table I shows that gap-lookahead and reinsertion mainly improve robustness, with success dropping more than 10% when either is removed. Removing mode-cache causes a smaller success drop to 92.5%, consistent with its role as a warm start rather than a feasibility gate. Quick-pass mainly improves efficiency: disabling it increases the average number of simulations from 121.5 to 208.2 and the planning time from 10.3 s to 23.6 s.

6) *Scalability and Robustness Analysis*: The generalization of PushAround is evaluated under obstacle density, robot-team size, and physics-model mismatch. All cases use the same 8×8 m scalability workspace with randomly generated movable objects of diverse shapes. Obstacle-density scalability varies the number of movable objects M , robot-team scalability varies the number of pushing robots N , and physics robustness is evaluated by planning with nominal parameters and executing under perturbed physics parameters. Specifically, the execution model introduces $\pm X\%$ variations in mass, friction, force limits, and contact dynamics. Table II summarizes representative results, while Fig. 8 and Fig. 9

TABLE II
REPRESENTATIVE SCALABILITY AND ROBUSTNESS RESULTS.

Case	M	N	Model	Succ.(%)	PT(s)
Base	10	2	Nom.		
Dense	45	2	Nom.		
Team	25	$2 \rightarrow 4$	Nom.		
Physics	25	2	$\pm X\%$		

M : number of movable obstacles; N : number of robots; Model: execution physics setting; Nom.: nominal model; $\pm X\%$: physics-parameter perturbation; Succ.: success rate; PT : planning time.

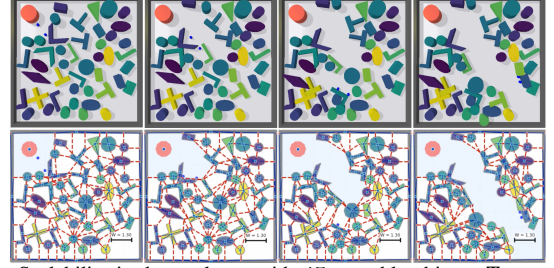


Fig. 8. Scalability in dense clutter with 45 movable objects. **Top**: snapshots during the path clearing of 24.5 s. **Bottom**: WCCG overlays.

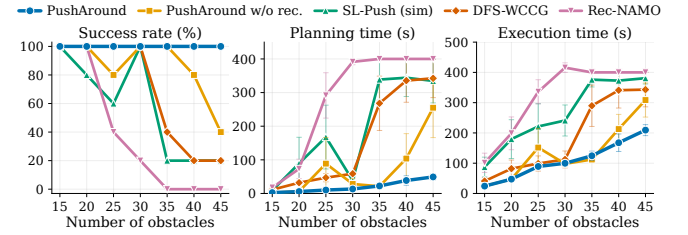


Fig. 9. Density scaling in the 8×8 m workspace. Comparison with baseline methods under increasing movable-obstacle density demonstrates the scalability of PushAround in dense clutter.

provide detailed evaluations of dense clutter and obstacle-number scaling.

B. Hardware Experiments

1) *System Description*: Experiments are conducted in a $5 \text{ m} \times 6 \text{ m}$ workspace assembled from interlocking $0.6 \text{ m} \times 0.6 \text{ m}$ foam mats, as illustrated in Fig. 10. Each trial employs 2 UGVs and 6 movable obstacles represented by cardboard boxes covered with colored paper for visual distinction. Both robots and movables are equipped with three to four motion-capture markers. A motion-capture system streams global poses to PyBullet in real time, enabling policy execution and visualization. The two Mecanum UGVs are controlled through planar velocity commands, while the optimized contact forces are used only for feasibility evaluation rather than direct motor control. Motion-capture feedback enables execution monitoring and online replanning. Movables are randomly placed for each trial.

2) *Results*: Two representative real-world runs are shown in Fig. 10. In Scenario 1, PIHS breaks five candidate gaps within 6.5 s, using 12 node expansions, 49 visited system-state snapshots, and a maximum search depth of 6. The four-task schedule first pushes and rotates the yellow L-shaped object to open a gap near an immovable obstacle. It then pushes the green and orange rectangular objects in opposite lateral directions, progressively expanding the reachable W-clearance region. After the green cylinder is pushed to the

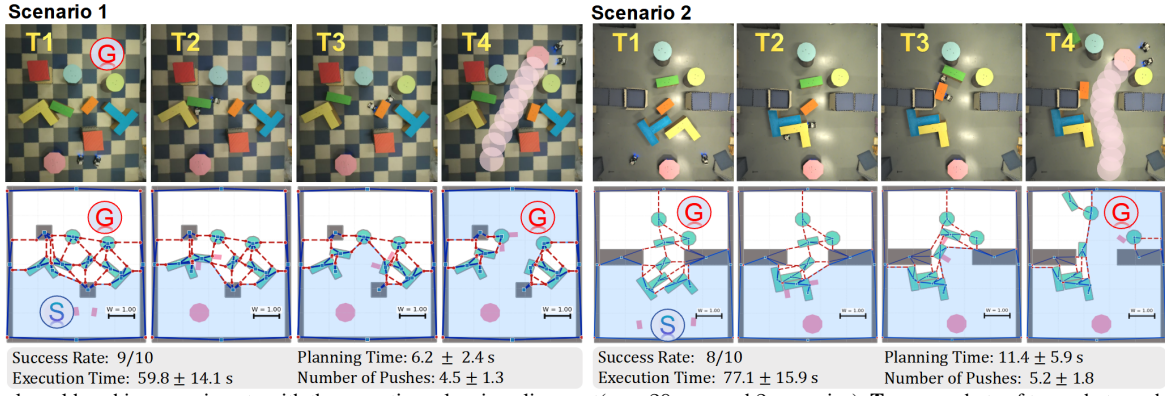


Fig. 10. Real-world pushing experiments with the execution–planning alignment (over 20 runs and 2 scenarios). **Top:** snapshots of two robots pushing obstacles in a 5×6 m workspace with movable objects and fixed boundaries. **Bottom:** WCCG overlays with the start face in blue.

right, a complete W -clear path is established at $t = 52$ s. In Scenario 2, the proposed method succeeds in 8/10 trials by generating an average of 5.2 pushes. The planning phase takes 11.4 s on average, and first selects a direct push on the blue L-shaped object, which also moves the adjacent T-shaped block and opens the first gap into the narrow passage. **This neighboring-object motion is not a separately encoded action but an emergent outcome of the coupled rigid-body rollout in EvalSim. The resulting snapshot is used to update the WCCG and continue replanning.** The complete execution takes 77.1 s on average with 15.9 s deviations. The main deviations arise from direction-dependent force limits, drifting, and Mecanum-wheel slippage near 45° motion directions, which are compensated by online replanning.

V. CONCLUSION

This work presents a multi-robot framework PushAround for path clearing in unstructured environments using a hybrid search algorithm that jointly plans obstacles selection, contact points, and pushing forces efficiently. The method demonstrates hardware feasibility and uses state-feedback replanning to recover from observed execution deviations. Future directions include obstacle states estimation without external sensing, uncertain masses, and limited robot visibility for real-world applications.

REFERENCES

- [1] L. Liu, X. Wang, X. Yang, H. Liu, J. Li, and P. Wang, "Path planning techniques for mobile robots: Review and prospect," *Expert Systems with Applications*, vol. 227, p. 120254, 2023.
- [2] M. Stilman and J. J. Kuffner, "Navigation among movable obstacles: Real-time reasoning in complex environments," *International Journal of Humanoid Robotics*, vol. 2, no. 04, pp. 479–503, 2005.
- [3] M. Stilman, J.-U. Schamburek, J. Kuffner, and T. Asfour, "Manipulation planning among movable obstacles," in *Proceedings 2007 IEEE international conference on robotics and automation*, 2007, pp. 3327–3332.
- [4] L. Yao, V. Modugno, A. M. Delfaki, Y. Liu, D. Stoyanov, and D. Kanoulas, "Local path planning among pushable objects based on reinforcement learning," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024, pp. 3062–3068.
- [5] Z. Ren, B. Suvonov, G. Chen, B. He, Y. Liao, C. Fermuller, and J. Zhang, "Search-based path planning in interactive environments among movable obstacles," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 533–539.
- [6] J. J. Weeda, S. Bakker, G. Chen, and J. Alonso-Mora, "Pushing through clutter with movability awareness of blocking obstacles," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 512–518.
- [7] Z. Tang, Y. Zhang, and M. Guo, "Pushingbots: Collaborative pushing via neural accelerated combinatorial hybrid optimization," *IEEE Transactions on Robotics*, vol. 42, pp. 579–599, 2026.
- [8] S. Goyal, A. Ruina, and J. Papadopoulos, "Limit surface and moment function descriptions of planar sliding," in *IEEE International Conference on Robotics and Automation (ICRA)*, 1989, pp. 794–795.
- [9] K. M. Lynch, H. Maekawa, and K. Tanie, "Manipulation and active sensing by pushing using tactile feedback," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, vol. 1, 1992, pp. 416–421.
- [10] J. Chen, M. Gauci, W. Li, A. Kolling, and R. Groß, "Occlusion-based cooperative transport with a swarm of miniature mobile robots," *IEEE Transactions on Robotics*, vol. 31, no. 2, pp. 307–321, 2015.
- [11] Y. Wang and C. W. De Silva, "Multi-robot box-pushing: Single-agent q-learning vs. team q-learning," in *IEEE/RSJ international conference on intelligent robots and systems*, 2006, pp. 3694–3699.
- [12] Y. Feng, C. Hong, Y. Niu, S. Liu, Y. Yang, and D. Zhao, "Learning multi-agent loco-manipulation for long-horizon quadrupedal pushing," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 14 441–14 448.
- [13] Y.-C. Lin, B. Ponton, L. Righetti, and D. Berenson, "Efficient humanoid contact planning using learned centroidal dynamics prediction," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2019, pp. 5280–5286.
- [14] Q. Rouxel, S. Ivaldi, and J.-B. Mouret, "Multi-contact whole-body force control for position-controlled robots," *IEEE Robotics and Automation Letters*, vol. 9, no. 6, pp. 5639–5646, 2024.
- [15] T. Xue, H. Girgin, T. S. Lembono, and S. Calinon, "Demonstration-guided optimal control for long-term non-prehensile planar manipulation," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2023, pp. 4999–5005.
- [16] D. M. Saxena and M. Likhachev, "Planning for complex non-prehensile manipulation among movable objects by interleaving multi-agent pathfinding and physics-based simulation," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2023, pp. 8141–8147.
- [17] K. Ren, G. Wang, A. S. Morgan, L. E. Kavraki, and K. Hang, "Object-centric kinodynamic planning for nonprehensile robot rearrangement manipulation," *IEEE Transactions on Robotics*, vol. 41, pp. 5761–5780, 2025.
- [18] R. Ni and A. H. Qureshi, "Progressive learning for physics-informed neural motion planning," in *Robotics: Science and Systems*, 2023.
- [19] Y. Liu, R. Ni, and A. H. Qureshi, "Physics-informed neural mapping and motion planning in unknown environments," *IEEE Transactions on Robotics*, vol. 41, pp. 2200–2212, 2025.
- [20] K. N. McGuire, G. C. H. E. de Croon, and K. Tuyls, "A comparative study of bug algorithms for robot navigation," *Robotics and Autonomous Systems*, vol. 121, p. 103261, 2019.
- [21] E. Coumans and Y. Bai, "Pybullet, a python module for physics simulation for games, robotics and machine learning," <http://pybullet.org>, 2016–2019.